



**Waiting for everyone
to join**

**TU257
Information Systems
Marisa Llorens**

Week 10 Information Systems

TU257

Agenda

- Introduction to MySQL
 - History
 - Description
 - Architecture
 - Data Types
- SQL code (continued)

MySQL: Brief Introduction

- Free SQL (Structured Query Language) Database Server
- MySQL is free and open-source software under the terms of the GNU General Public License, and is also available under a variety of proprietary licenses
- MySQL was created by a Swedish company, MySQL AB. Original development of MySQL began in 1994. The first version of MySQL appeared on 23 May 1995

MySQL: Brief Introduction

- Sun Microsystems acquired MySQL AB in 2008
- Oracle acquired Sun Microsystems on 27 January 2010
- The day Oracle announced the purchase of Sun, one of the founders of MySQL forked MySQL, launching MariaDB

MySQL: Storage Engine Architecture

- A key strength of MySQL is its unique **storage engine architecture**.
 - Storage Engines **separate** the **processing** of the **SQL** statements from the actual **storage/retrieval** of the data.

MySQL: Storage Engine Architecture

- A key strength of MySQL is its unique **storage engine architecture**.
 - By having separate tiers of the architecture to handle these two tasks, a client program is effectively removed from any storage considerations i.e. MySQL programs and statements written against a table that was created with one storage engine will work exactly the same (without change) against the same table stored with a different storage engine.

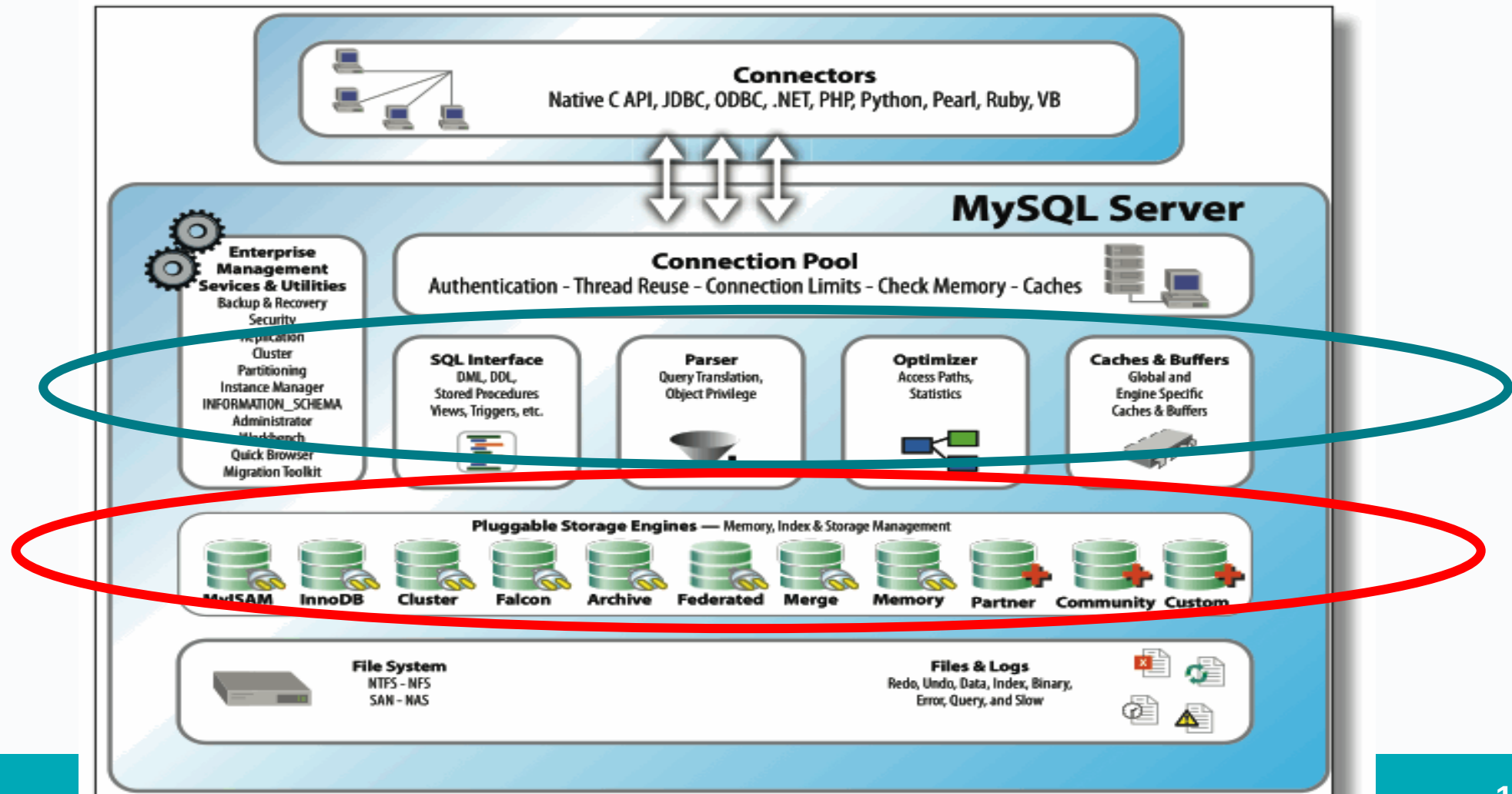
MySQL: Storage Engine Architecture (cont'd)

- MySQL has an **SQL tier** that performs the actual processing of the **SQL** statement. It...
 - **parses** the statement,
 - **validates** the syntax,
 - **builds** a query plan,
 - **optimises** the query plan and then
 - **executes** the query plan.

MySQL: Storage Engine Architecture (cont'd)

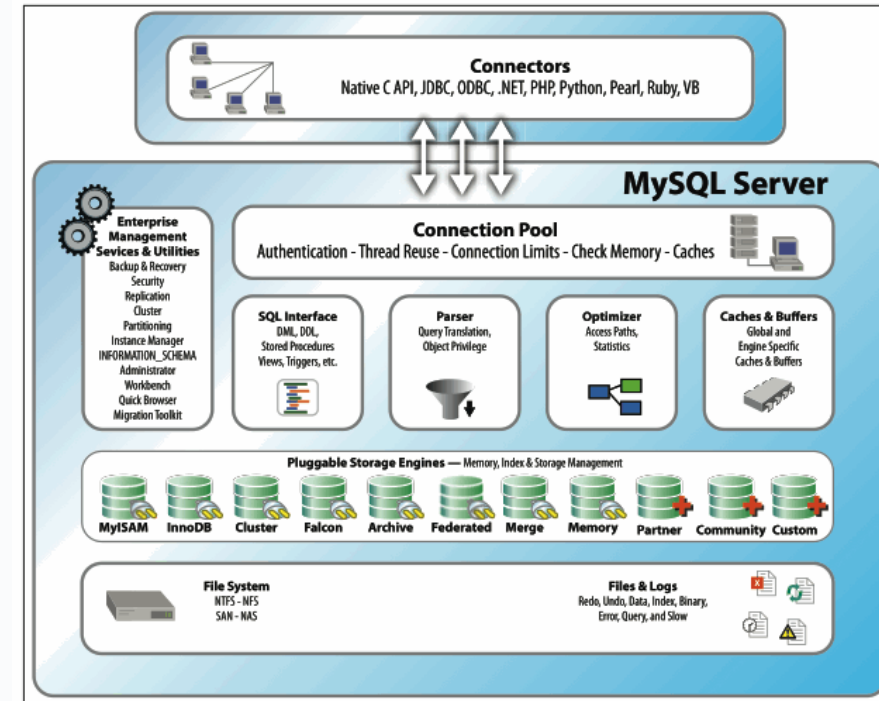
- It's only when the query plan is executed that the **storage tier** of MySQL takes over and actually
 - **retrieves**,
 - **stores**, or
 - **alters** the data as directed by the query plan.
- In this way, the **SQL** tier has **no dependencies** on which storage engine is actually managing the database table.

MySQL: Storage Engine Architecture (cont'd)



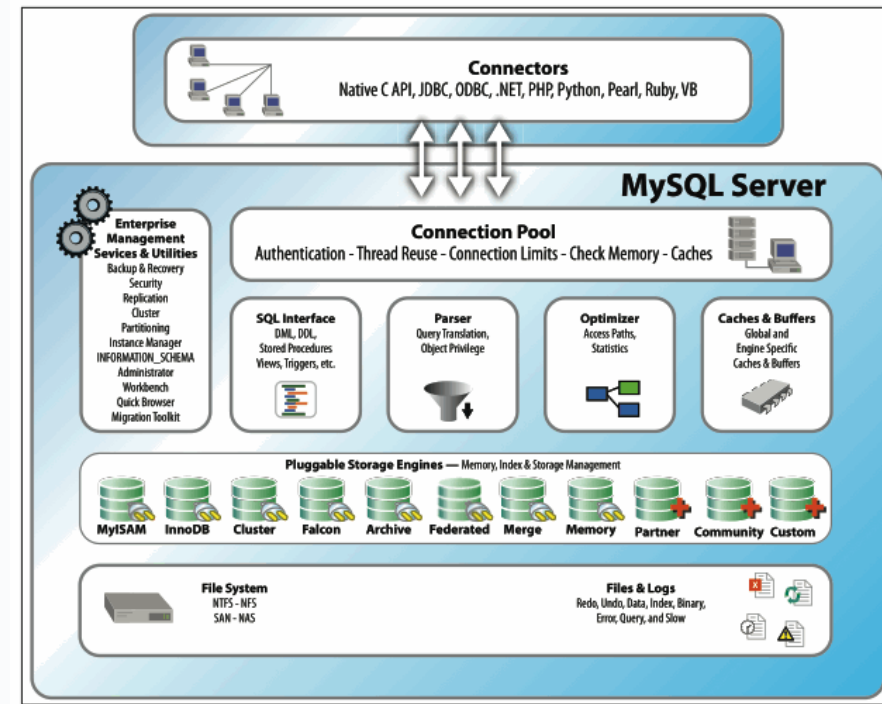
MySQL: Storage Engine Architecture (cont'd)

- This architecture allows the **implementation of specialised storage engines** to meet a number of divergent requirements.
- The **granularity** of the storage engine selection is at the table level
 - so different storage engines can be implemented to satisfy different requirements within the same database schema.



MySQL: Storage Engine Architecture (cont'd)

- Storage engines have a number of differentiators including:
 - Concurrency/Locking
 - Transaction Support
 - Index Support
 - Memory Caches
 - Performance Aids



When you omit the ENGINE option, the **default storage engine** is used. The default engine is InnoDB in **MySQL 8.0**.

MySQL: Storage Engine Architecture (cont'd)

SHOW ENGINES;

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)] [[Refresh](#)]

Extra options

Engine	Support	Comment	Transactions	XA	Savepoints
CSV	YES	Stores tables as CSV files	NO	NO	NO
MRG_MyISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary...	NO	NO	NO
Aria	YES	Crash-safe tables with MyISAM heritage. Used for i...	NO	NO	NO
MyISAM	YES	Non-transactional engine with good performance and...	NO	NO	NO
SEQUENCE	YES	Generated tables filled with sequential values	YES	NO	YES
InnoDB	DEFAULT	Supports transactions, row-level locking, foreign ...	YES	YES	YES
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO

MySQL: Data Types

Table Data Type	Description
INT	An integer from -2,147,483,648 to 2,147,483,647
DECIMAL	A floating point number that can specify the number of permissible digits e.g. DECIMAL(3,2) permits -999.99 to 999.99
DOUBLE	A long double-precision floating point number.
DATE	A date in the YYYY-MM-DD format.
TIME	A time in the HH:MM:SS format.
DATETIME	A combined date and time in the format YYYY-MM-DD HH:MM:SS
YEAR	A year 1901-2155 in either YY or YYYY format.
TIMESTAMP	Automatic date and time of last record entry.
CHAR()	A string of defined fixed length up to 255 characters e.g. CHAR(100) pads a smaller string to make it 100 characters long.
VARCHAR()	A string of variable length up to 255 characters long that is stored without padding.
TEXT	A string up to 65,535 characters long.
BLOB	A binary type of variable data.
ENUM	A single string value from a defined list e.g. ENUM("red", "green", "blue") allows entry of any one of these three colours only.
SET	A string of multiple strings from a defined list e.g. SET("red", "green", "blue") allows entry of any one, or more, of these colours.

MySQL: Numeric Data Types

- **Whole Numbers:**
 - **TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT:** Require 8, 16, 24, 32, and 64 bits of storage space, respectively. They can store values from $-2^{(n-1)}$ to $2^{(n-1)}-1$, where **N** is the number of bits of storage space they use.
 - Use **UNSIGNED** when you don't need negative numbers, this doubles the bits of storage space. **BIGINT** is not needed for **AUTO_INCREMENT**, **INT UNSIGNED** stores 4.3 billion values.
 - **INT(1)** does not mean 1 digit: The number in parentheses is the **ZEROFILL** argument, and specifies the number of characters some tools reserve for display purposes. For storage and computational purposes, **INT(1)** is identical to **INT(20)**.
 - Integer data types work best for primary key data types.

MySQL: Numeric Data Types

- **Real Numbers:**
 - **FLOAT, DOUBLE:** Supports approximate calculations with standard floating-point math.
 - **DECIMAL:** use **DECIMAL** when you need exact results, always use for monetary/currency fields.
- **Other:**
 - **BIT:** to store 0,1 values. **BIT(1)** stores 1 bit, **BIT(2)** stores 2... max is 64.

MySQL: Character Data Types

- The **CHAR** and **VARCHAR** types are declared with a length that indicates the maximum number of characters to store.
 - **VARCHAR(n)** stores variable-length character strings.
 - **VARCHAR** uses only as much space as it needs, which helps performance because it saves disk space.
 - Use when the maximum column length is larger than the average length. Try to keep **VARCHAR** Max Length short.

MySQL: Character Data Types

- **CHAR(n)** is fixed-length:
 - MySQL allocates enough space for the specified number of characters.
 - Useful to store very short strings, when all the values are nearly the same length, and for data that's changed frequently.

MySQL: NULL/NOT NULL

- **Use NOT NULL**
 - Always define columns as **NOT NULL** unless there is a very good reason not to do so:
 - can save up to a byte per column per row of data
 - nullable columns make indexes, index statistics, and value comparisons more complicated.

MySQL: Case Sensitivity & Identifiers

- Database names have the same case sensitivity as directories in your operating system.
 - Table names follow the same rules as filenames.
 - Everything else is case sensitive.
- All identifiers except aliases can be up to 64 characters long.
 - Aliases can be up to 255 characters long.

MySQL: Case Sensitivity & Identifiers

- Identifiers can contain most characters, but database names may not contain `/`, `\`, or `.` and table names cannot contain `.` or `/`.
- You can use reserved words for identifiers as long as you use quotes.

How MySQL Stores Data (by default)

- A MySQL server can store several databases.
- Databases are stored as directories:
 - Default is at...
 - /var/mysql *Mac OS*
 - C:\ProgramData\MySQL\MySQL Server 8.0\Data *Windows*
- Tables are stored as .ibd files inside each database (InnoDB).

Basic SQL statements

- ➔ **CREATE** – Create a new data structure
- ➔ **DROP** – Remove a data structure
- ➔ **ALTER** – Modify a data structure

- ➔ **SELECT** - Read one or more rows from a table
- ➔ **INSERT** - One or more rows into a table
- ➔ **DELETE** - One or more rows from a table
- ➔ **UPDATE** - Change the column values in a row

SQL: Retrieve Records using criteria

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000
.....				

Open world.db database (SQLite)

SQL: Retrieve Records using criteria

- Show the name, continent and population of all countries.
- Show the name for the countries that have a population of at least 200 million.
- Show the name and per capita gdp for countries with population at least 200 million.

SQL: Null & Three-valued logic (3VL)

- Comparisons combined using **AND**, **OR** and **NOT**.
- Comparisons are (**Attr** *op* const) or (**Attr1** *op* **Attr2**), where *op* is one of...
- = > < >= <= <> !=
- **not equal** to (<>, !=)

SQL: Null & Three-valued logic (3VL)

- Comparison operations result in a value of 1 (TRUE), 0 (FALSE), or NULL.
- These operations work for both numbers and strings.
- The relational comparison operators can be used to compare not only scalar operands, but row operands.

SQL: Null & Three-valued logic (3VL)

Logical Conjunction

p AND q		p		
		True	False	Unknown
q	True	True	False	Unknown
	False	False	False	False
	Unknown	Unknown	False	Unknown

Logical Disjunction

p OR q		p		
		True	False	Unknown
q	True	True	True	True
	False	True	False	Unknown
	Unknown	True	Unknown	Unknown

Logical Equivalence

p = q		p		
		True	False	Unknown
q	True	True	False	Unknown
	False	False	True	Unknown
	Unknown	Unknown	Unknown	Unknown

Logical Negation

p	NOT p
True	False
False	True
Unknown	Unknown

Note: p and q represent logical states. Unknown also means Null.

MySQL: Table Retrieval Examples

- **OR**

```
SELECT studentName FROM students WHERE degree = 'BA' OR degree = 'BSc';
```

- **AND**

```
SELECT COUNT(regNo) FROM cars WHERE carMake="Ford" AND carModel="Fiesta";
```

MySQL: Table Retrieval Examples

- **NULL**

- No Value
- Can not use the usual comparison operators (>, =, != ...)
- Use **IS** or **IS NOT** operators to compare with e.g.

```
SELECT studentName  
FROM students  
WHERE projectID IS NULL;
```

SQL: Retrieve Records using AND/OR operators

- Show the name and population for Andorra and Angola.
- Show large countries in extension or population (area>2m, population>200m). Show name, population and area.
- Show the countries that are large in area and population. Show name, population and area.

FORMATTING THE OUTPUT

- Show the average **population** per continent formatting to 2 decimal places and "#,###,###.##".

```
SELECT continent, FORMAT(AVG(population),0) AS "Average  
Population" FROM countries GROUP BY continent;
```


ORDER BY using two columns

- **ORDER BY** (Sort query results)

```
SELECT * FROM countries WHERE continent = "Europe" ORDER BY area ASC;
```

```
SELECT * FROM countries WHERE continent = "Europe" ORDER BY area ASC, population DESC;
```

Joins

Introduction: Join Operations

- A *join* is the process of linking two tables together.
 - Using more than a single table of a database is usually essential.
 - You specify the tables to join in the **FROM** clause together with the type of join.
 - You should also specify a join condition to describe how the tables should be joined together.
- Why do we require JOIN Operations?
 - In a relational database, data is kept mostly normalized in tables.
 - Obtaining meaningful data may require combining multiple tables, for which we require JOIN.

Introduction: Join Operations

How does a JOIN operate?

The tables are joined using a condition.

The condition mostly involves the matching column(s).

Based on the matching conditions and the type of join we use, the results may vary.

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- CROSS JOIN

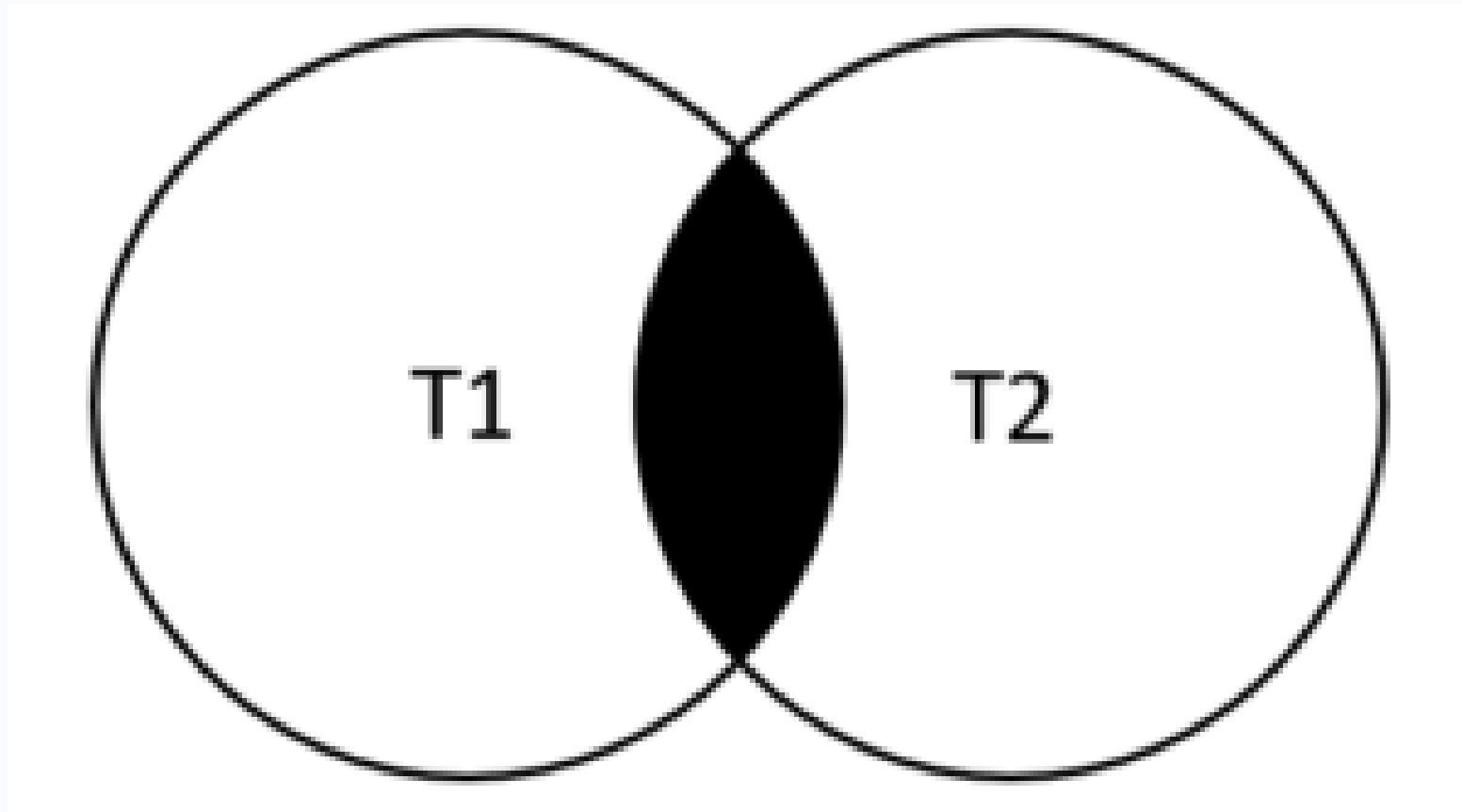
Join Operations **INNER** Join

The inner join matches each row in one table with every row in other tables and allows you to query rows that contain values from both tables.

The INNER JOIN is an optional clause of the SELECT statement. It appears immediately after the FROM clause.

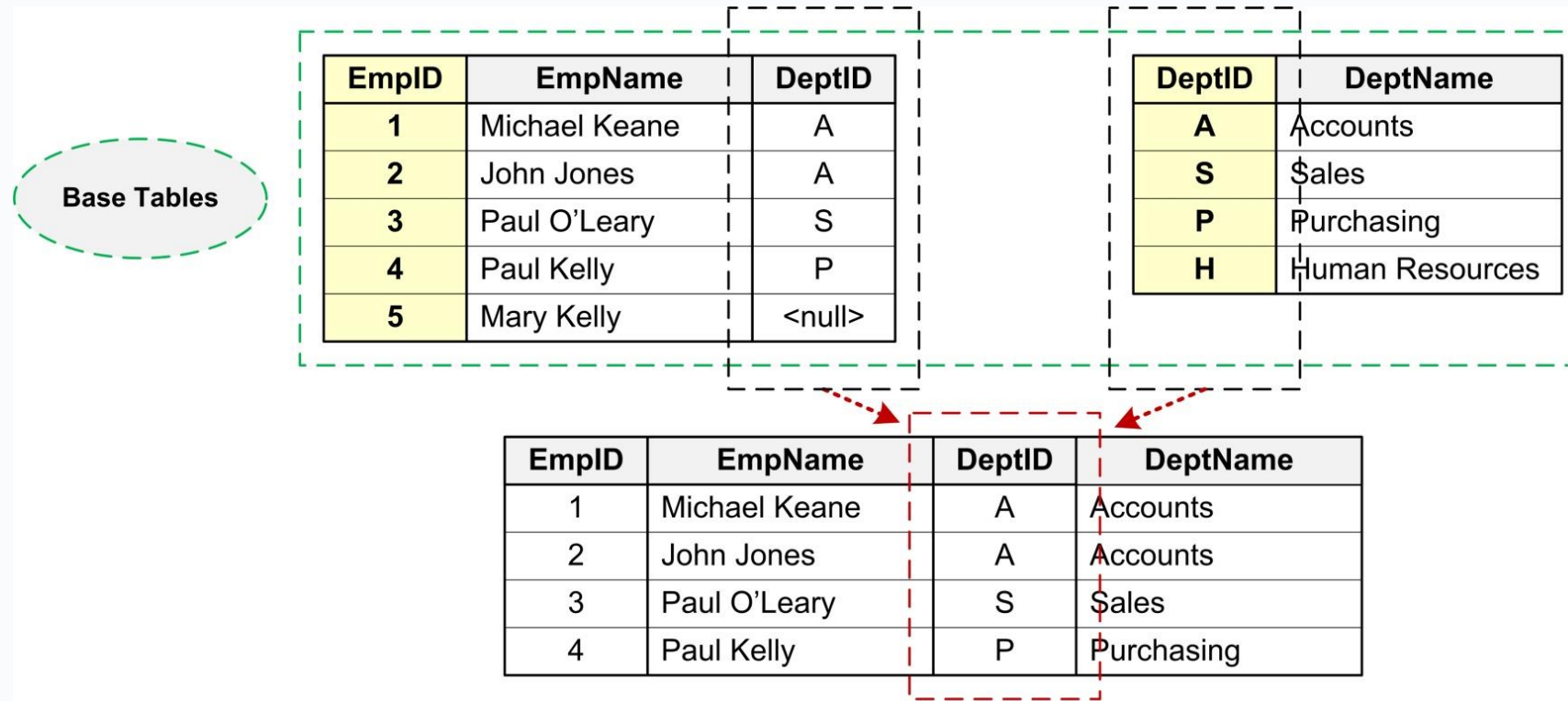
```
SELECT select_list FROM t1  
INNER JOIN t2 ON join_condition
```

Join Operations **INNER** Join



Join Operations: The **INNER** Join

- Example of an **INNER** (Natural) Join.
- It is based on equality of values in the joined columns.



Open
mycompany.db

Join Operations: The **INNER** Join (SQLite)

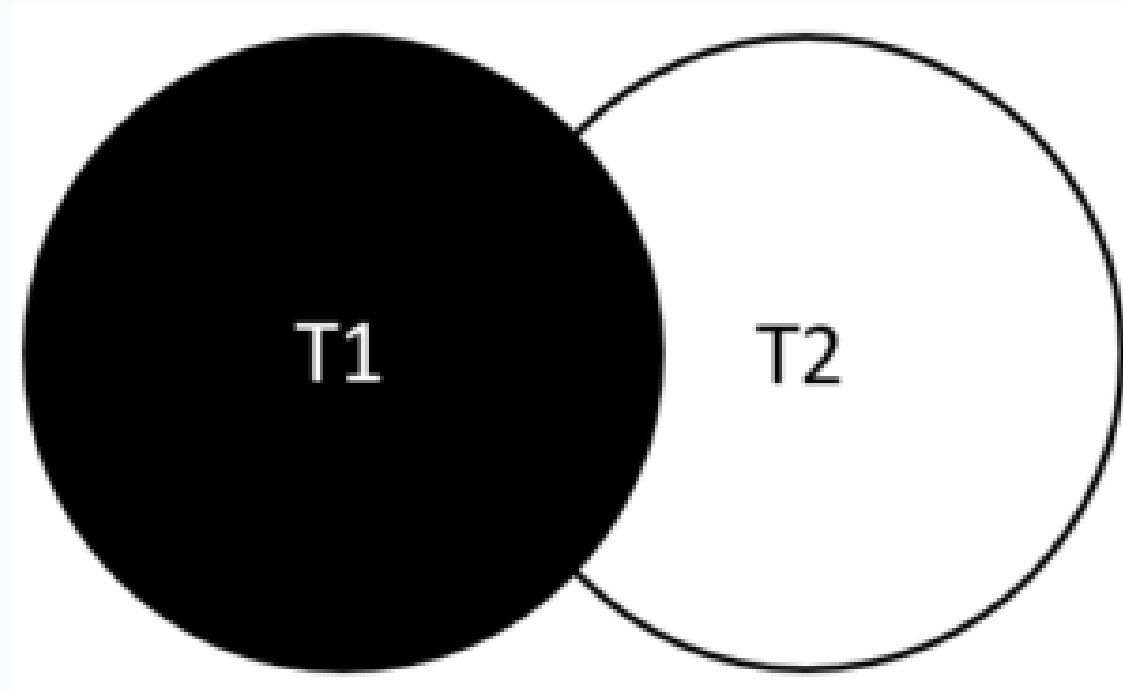
```
SELECT employees.employeeID, employees.employeeName,  
employees.DeptID, departments.DeptName  
FROM employees INNER JOIN departments ON employees.DeptID=departments.DeptID;
```

	employeeID	employeeName	DeptID	DeptName
1	1	Michael Keane	A	Accounts
2	2	John Jones	A	Accounts
3	3	Paul O'Leary	S	Sales
4	4	Paul Kelly	P	Purchasing

Join Operations: The **LEFT** Join

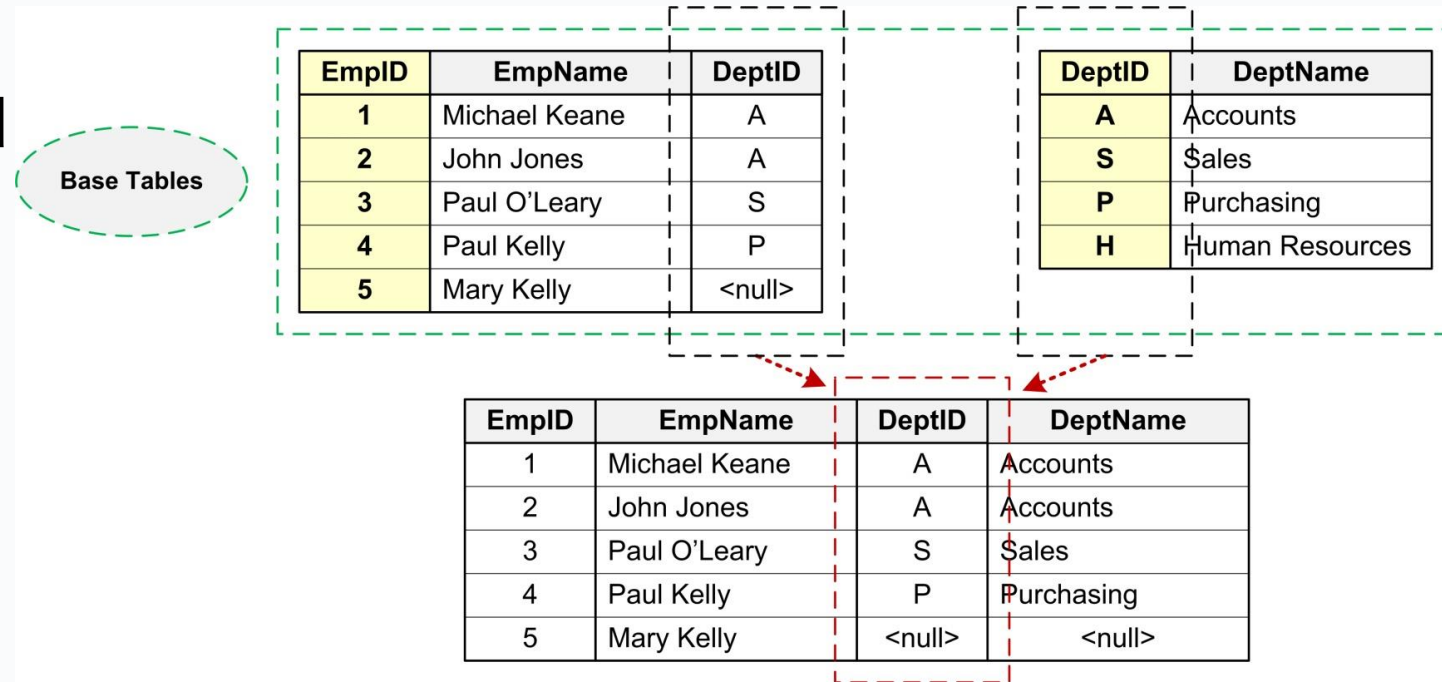
- The LEFT JOIN matches each row from the left table with every row from the right table based on the join condition.
- The left-side table retains all data even if there is no matching row.
- If there is no match, the columns of the row from the right table will contain NULL.

Join Operations: The **LEFT** Join



Join Operations: The **LEFT** Join

- This shows all employees, even if they are not allocated to a department, together with details of departments which have employees.
- The employees table is on the left-hand side of the join expression.



SQL: **Where** *tb/*Employees.DeptID = *tb/*Departments.DeptID

Join Operations: The **LEFT** Join

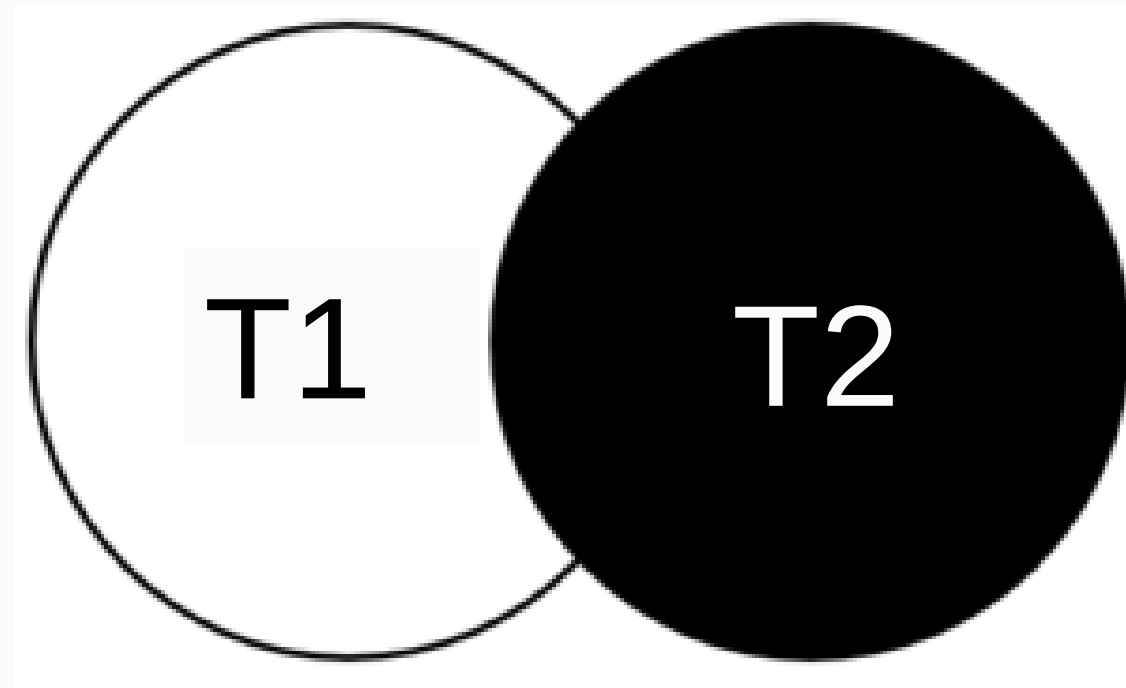
```
SELECT  employees.empID, employees.empName,  
        departments.DeptName  
FROM    employees LEFT JOIN departments  
ON      employees.DeptID=departments.DeptID;
```

	EmpID	EmpName	DeptName
1	1	Michael Keane	Accounts
2	2	John Jones	Accounts
3	3	Paul OLeary	Sales
4	4	Paula Kelly	Purchasing
5	5	Mary Kelly	NULL

Join Operations: The **RIGHT** Join

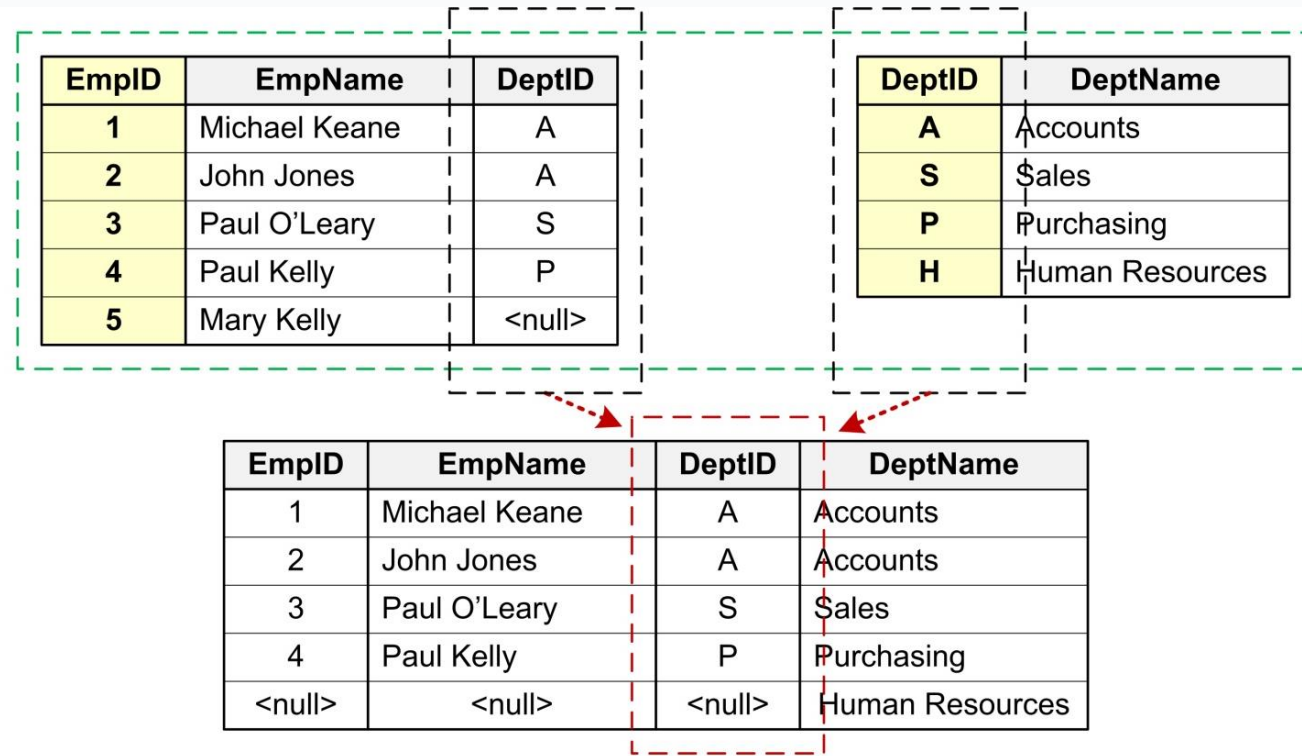
- The RIGHT JOIN matches each row from the right table with every row from the left table based on the join condition.
- The right-side table retains all data even if there is no matching row.
- If there is no match, the columns of the row from the left table will contain NULL.

Join Operations: The **RIGHT** Join



Join Operations: The RIGHT Join

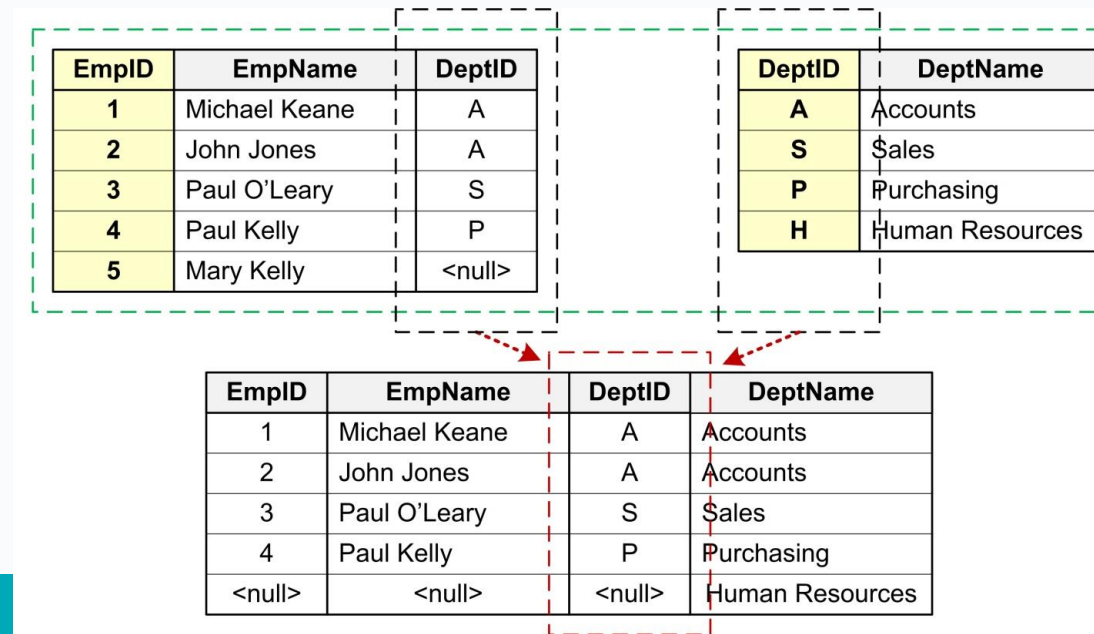
- This view shows all departments, even if they have no employees, with details of employees who are allocated to a department.



SQL: **Where** *tb*/Employees.DeptID = *tb*/Departments.DeptID

Join Operations: The RIGHT Join

```
SELECT  employees.empID,employees.empName,departments.DeptName  
FROM employees RIGHT JOIN departments  
ON employees.DeptID=departments.DeptID;
```



SQL: **Where** *tb/*Employees.DeptID = *tb/*Departments.DeptID

Join Operations: USING

- If the join_condition uses the equal operator and the joined columns of both tables have the same name, you can use the USING syntax:

```
SELECT  
employees.empName,departments.DeptName  
FROM employees LEFT JOIN departments  
USING( DeptID);
```

Join Operations: CROSS JOIN

- A **cross join** is used when you wish to create a combination of every row from two tables. All row combinations are included in the result; this is commonly called **cross product join**.
- A common **use** for a **cross join** is to create obtain all combinations of items, such as colors and sizes

```
SELECT employees.empID,employees.empName,departments.DeptName  
FROM employees CROSS JOIN departments ;
```

Join Operations: CROSS JOIN

employeeID	employeeName	DeptName
1	Michael Keane	Accounts
1	Michael Keane	Sales
1	Michael Keane	Purchasing
1	Michael Keane	Human Resources
2	John Jones	Accounts
2	John Jones	Sales
2	John Jones	Purchasing
2	John Jones	Human Resources
3	Paul O'Leary	Accounts
3	Paul O'Leary	Sales
3	Paul O'Leary	Purchasing
3	Paul O'Leary	Human Resources
4	Paul Kelly	Accounts
4	Paul Kelly	Sales
4	Paul Kelly	Purchasing
4	Paul Kelly	Human Resources

Not very useful in this example.

Join Operations: More than two tables

```
SELECT employees.empName, projects.projectName  
FROM(  
    employees INNER JOIN departments USING (DeptID)  
    INNER JOIN projects USING (DeptID)  
);
```

	EmpName	projectName
1	John Jones	ScienceFest
2	Michael Keane	ScienceFest
3	Paul OLeary	SalesPitch

Join Operations:

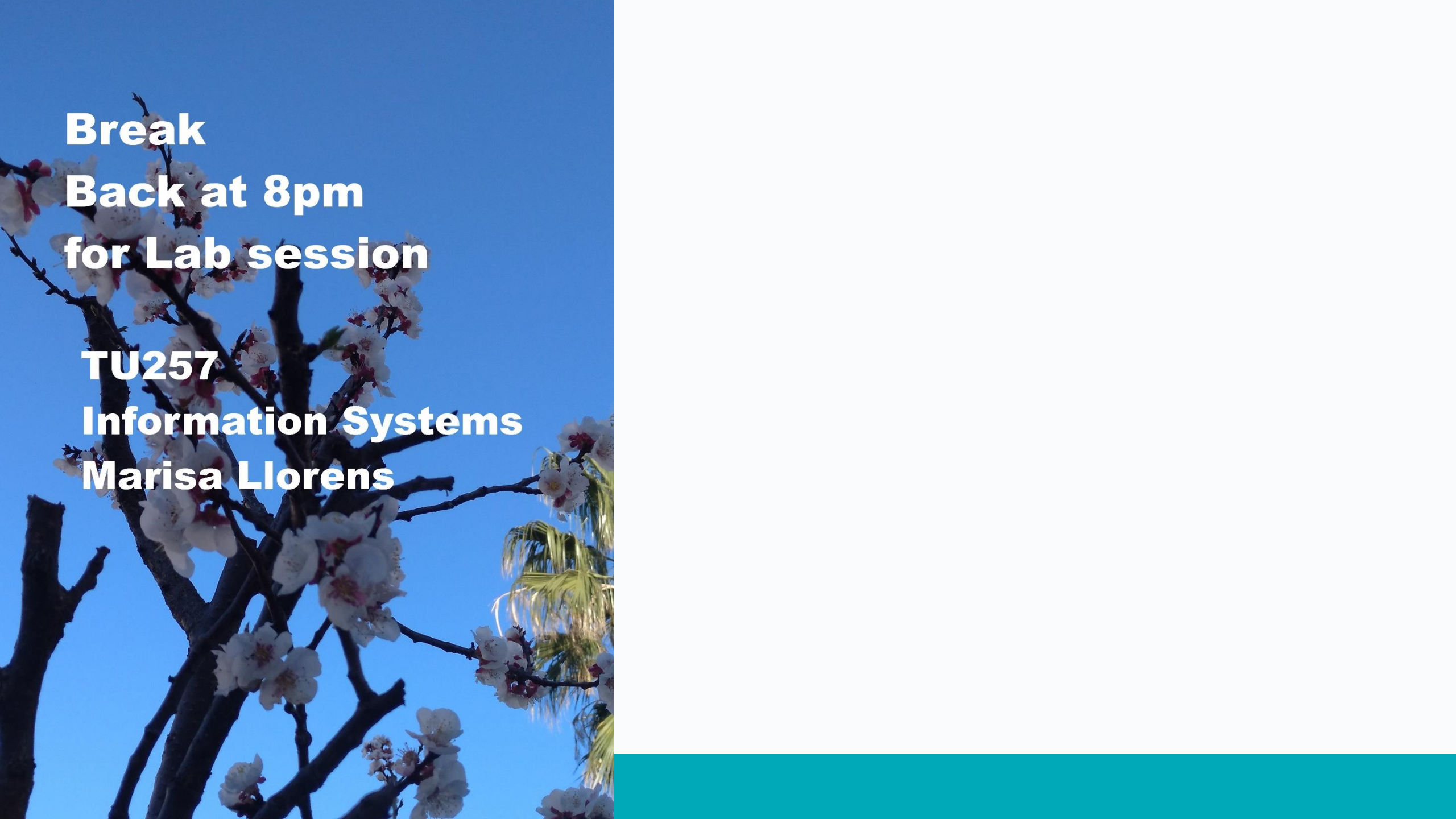
Adding a condition after JOIN

```
SELECT employees.empName, departments.DeptName FROM  
employees  
LEFT JOIN departments USING( DeptID) WHERE DeptID="S";
```

```
SELECT employees.empName, projects.projectName  
FROM(  
    employees INNER JOIN departments USING (DeptID)  
    INNER JOIN projects USING (DeptID)  
)  
WHERE DeptID='S';
```

Thanks

- Edmund Nevin for earlier versions of these lecture notes



**Break
Back at 8pm
for Lab session**

**TU257
Information Systems
Marisa Llorens**

APPENDIX

MySQL: Storage Engine Architecture

- MySQL storage engines can be broken down into two broad categories, ***specialty*** and ***general purpose*** storage engines.
 - **Specialty** storage engines include the following:
 - **Memory**: very fast, no persistence; typically used for temporary tables
 - **Archive**: supports high write bandwidth, no updates, slower reads, and limited indexing
 - **BrightHouse**: highly compressed, used for multi-terabyte data warehouses
 - **NitroEDB**: supports high-volume real-time analysis

APPENDIX

MySQL: Storage Engine Architecture

- MySQL storage engines can be broken down into two broad categories, ***specialty*** and ***general purpose*** storage engines.
 - **General purpose** storage engines include the following
 - **MyISAM**: default storage engine for MySQL.; fast but does not support transactions
 - **Maria**: crash-recoverable version of MyISAM
 - **InnoDB**: widely-used storage engine that supports transactions
 - **IBMDB2I**: transactional engine to enable access to MySQL data through traditional DB2 for i interfaces